

Библиотека за представяне, обработка и анализ на данни Pandas.

доц. д-р Гергана Спасова
Кат. „Компютърни науки и технологии“

Инсталиране и използване на Pandas

Инсталирането на Pandas изисква NumPy да бъде предварително инсталиран. След като Pandas бъде инсталиран, може да се импортира и да се провери версията му:

```
In[1]: import pandas  
pandas.__version__  
Out[1]: '0.18.1'
```

Точно както се импортира NumPy под псевдонима np, се импортира и Pandas под псевдонима pd:

```
In[2]: import pandas as pd
```

Представяне на обекти в Pandas

На съвсем основното ниво обектите на Pandas могат да се разглеждат като подобрени версии на структурирани масиви на NumPy, в които редовете и колоните се идентифицират с етикети, различни от обикновени индекси с цели числа. Pandas предоставя множество полезни инструменти, методи и функционалности върху структурите на основните данни. Трите основни структури от данни на Pandas са: Series, DataFrame и Index.

```
In[1]: import numpy as np  
import pandas as pd
```

Обектът Series на Pandas

Series е едномерен масив от индексирани данни. Той може да бъде създаден от лист или масив, както следва:

```
In [2]: data = pd.Series([0.25, 0.5, 0.75, 1.0])  
data
```

```
Out[2]: 0    0.25  
        1    0.50  
        2    0.75  
        3    1.00  
        dtype: float64
```

Series обхваща както последователност от стойности, така и последователност от индекси, до които може да се осъществи достъп със стойностите и атрибутите на индекса. Стойностите са масив NumPy:

```
In [3]: data.values
```

```
Out[3]: array([ 0.25,  0.5 ,  0.75,  1.  ])
```

Обектът Series на Pandas

Индексът е подобен на масив:

```
In [4]: data.index
```

```
Out[4]: RangeIndex(start=0, stop=4, step=1)
```

Подобно на масив NumPy, данните могат да бъдат достъпни от свързания индекс чрез квадратна скоба:

```
In [5]: data[1]
```

```
Out[5]: 0.5
```

```
In [6]: data[1:3]
```

```
Out[6]: 1    0.50  
        2    0.75  
        dtype: float64
```

Обектът Series на Pandas

Пример: Използване на низове като индекс:

```
In [7]: data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                        index=['a', 'b', 'c', 'd'])  
data
```

```
Out[7]: a    0.25  
       b    0.50  
       c    0.75  
       d    1.00  
       dtype: float64
```

Пример: Достъп до обект:

```
In [8]: data['b']
```

```
Out[8]: 0.5
```

Обектът Series на Pandas

Пример: Използване на непрекъснати или непоследователни индекси:

```
In [9]: data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                        index=[2, 5, 3, 7])  
data
```

```
Out[9]: 2    0.25  
        5    0.50  
        3    0.75  
        7    1.00  
dtype: float64
```

```
In [10]: data[5]
```

```
Out[10]: 0.5
```

Изграждане на Series обекти

Данните могат да бъдат списък или масив NumPy като индексът по подразбиране е целочислена последователност:

```
In [14]: pd.Series([2, 4, 6])
```

```
Out[14]: 0    2  
         1    4  
         2    6  
         dtype: int64
```

Данните могат да бъдат скалярни, за запълване на индекс:

```
In [15]: pd.Series(5, index=[100, 200, 300])
```

```
Out[15]: 100    5  
         200    5  
         300    5  
         dtype: int64
```

Изграждане на Series обекти

Данните могат да бъдат речник, в който индексът по подразбиране е на сортираните речникови ключове:

```
In [16]: pd.Series({2:'a', 1:'b', 3:'c'})
```

```
Out[16]: 1    b  
         2    a  
         3    c  
         dtype: object
```

Във всеки случай индексът може да бъде изрично зададен, ако се предпочита различен резултат:

```
In [17]: pd.Series({2:'a', 1:'b', 3:'c'}, index=[3, 2])
```

```
Out[17]: 3    c  
         2    a  
         dtype: object
```

DataFrame обект

Series е аналог на едномерен масив с гъвкави индекси, а DataFrame е аналог на двумерен масив с гъвкави индекси както по редове, така и по колони.

Пример: Създаване на серия, изброяваща площта на всеки от петте града:

```
In [18]: area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,  
                      'Florida': 170312, 'Illinois': 149995}  
area = pd.Series(area_dict)  
area
```

```
Out[18]: California    423967  
Florida      170312  
Illinois     149995  
New York     141297  
Texas        695662  
dtype: int64
```

DataFrame обект

Може да се използва речник, за да се изгради единичен двуизмерен обект, съдържащ тази информация:

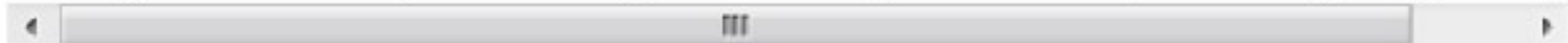
```
In [19]: states = pd.DataFrame({'population': population,  
                                'area': area})  
states
```

```
Out[19]:
```

	area	population
California	423967	38332521
Florida	170312	19552860
Illinois	149995	12882135
New York	141297	19651127
Texas	695662	26448193

```
In [20]: states.index
```

```
Out[20]: Index(['California', 'Florida', 'Illinois', 'New York', 'Texas'], dtype='obj'
```



DataFrame обект

Освен това DataFrame има атрибут `columns`, който е обект на `index`, съдържащ етикетите на колоните:

```
In [21]: states.columns
```

```
Out[21]: Index(['area', 'population'], dtype='object')
```

По този начин DataFrame може да се разглежда като двуизмерен NumPy масив, където както редове, така и колони имат обобщен индекс за достъп до данните.

Изграждане на обекти на DataFrame

- От един обект от серия

DataFrame е колекция от серийни обекти. DataFrame с една колона може да бъде конструиран от една серия:

```
In [23]: pd.DataFrame(population, columns=['population'])
```

```
Out [23]:
```

	population
California	38332521
Florida	19552860
Illinois	12882135
New York	19651127
Texas	26448193

Изграждане на обекти на DataFrame

- **От списък с речници**

Всеки списък с речници може да бъде превърнат в DataFrame.

```
In [24]: data = [{'a': i, 'b': 2 * i}
               for i in range(3)]
          pd.DataFrame(data)
```

```
Out[24]:
```

	a	b
0	0	0
1	1	2
2	2	4

Дори ако някои ключове в речника липсват, Pandas ще ги попълни със стойности NaN.

```
In [25]: pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

```
Out[25]:
```

	a	b	c
0	1.0	2	NaN
1	NaN	3	4.0

Изграждане на обекти на DataFrame

- От речник на Series обекти

DataFrame може да бъде конструиран и от речник на Series обекти:

```
In [26]: pd.DataFrame({'population': population,  
                        'area': area})
```

```
Out [26]:
```

	area	population
California	42396738332521	
Florida	17031219552860	
Illinois	14999512882135	
New York	14129719651127	
Texas	69566226448193	

Изграждане на обекти на DataFrame

- От двумерен масив NumPy

Може да се създаде DataFrame с всякакви посочени имена на колони и индекси. Ако не се уточни, ще се използва целочислен индекс:

```
In [27]: pd.DataFrame(np.random.rand(3, 2),  
                      columns=['foo', 'bar'],  
                      index=['a', 'b', 'c'])
```

Out [27]:

	foo	bar
a	0.865257	0.213169
b	0.442759	0.108267
c	0.047110	0.905718

Изграждане на обекти на DataFrame

- От структуриран масив NumPy

DataFrame работи подобно на структуриран масив и може да бъде създаден директно от него:

```
In [28]: A = np.zeros(3, dtype=[('A', 'i8'), ('B', 'f8')])  
A
```

```
Out[28]: array([(0, 0.0), (0, 0.0), (0, 0.0)],  
              dtype=[('A', '<i8'), ('B', '<f8')])
```

```
In [29]: pd.DataFrame(A)
```

```
Out[29]:
```

	A	B
0	0	0.0
1	0	0.0
2	0	0.0

Операции с данни в Pandas

Една от съществените части на NumPy е способността да се извършват бързи операции по елементи, както с основна аритметика (събиране, изваждане, умножение и т.н.), така и с по-сложни операции (тригонометрични функции, експоненциални и логаритмични функции и др.). Pandas наследява голяма част от тази функционалност от NumPy и ufuncs.

Следващата таблица изброява Python оператори и техните еквивалентни Pandas обектни методи:

Python оператор	Pandas метод
+	add()
-	sub(), subtract()
*	mul(), multiply()
/	truediv(), div(), divide()
//	floordiv()
%	mod()
**	pow()

Операции между DataFrame и Series

Операциите между DataFrame и Series са подобни на операции между двумерен и едномерен NumPy масив.

```
In [15]: A = rng.randint(10, size=(3, 4))  
A
```

```
Out[15]: array([[3, 8, 2, 4],  
                [2, 6, 4, 8],  
                [6, 1, 3, 8]])
```

```
In [16]: A - A[0]
```

```
Out[16]: array([[ 0,  0,  0,  0],  
                [-1, -2,  2,  4],  
                [ 3, -7,  1,  4]])
```

Операции между DataFrame и Series

В Pandas конвенцията работи по подразбиране по редове:

```
In [17]: df = pd.DataFrame(A, columns=list('QRST'))  
df = df.iloc[0]
```

```
Out[17]:
```

	Q	R	S	T
0	0	0	0	0
1	-1	-2	2	4
2	3	-7	1	4

Операции между DataFrame и Series

Ако трябва да се работи по колона може да се използва:

```
In [18]: df.subtract(df['R'], axis=0)
```

```
Out[18]:
```

	Q	R	S	T
0	50	6	4	
1	40	2	2	
2	5	0	2	7

```
In [19]: halfrow = df.iloc[0, ::2]  
halfrow
```

```
Out[19]: Q      3  
         S      2  
         Name: 0, dtype: int64
```

```
In [20]: df - halfrow
```

```
Out[20]:
```

	Q	R	S	T
0	0.0	NaN	0.0	NaN
1	-1.0	NaN	2.0	NaN
2	3.0	NaN	1.0	NaN

Индексът в Pandas

Обектите Series и DataFrame съдържат изричен индекс, който позволява препращане и модифициране на данни. Този Index обект сам по себе си е интересна структура и може да се разглежда или като неизменяем масив, или като подреден набор.

Пример: Изграждане на Index от списък с цели числа:

```
In [30]: ind = pd.Index([2, 3, 5, 7, 11])  
ind
```

```
Out[30]: Int64Index([2, 3, 5, 7, 11], dtype='int64')
```